Human Verification: `Containers.Monoidal`

The Sequential Operator as a Monoidal Category

Machine-generated from `lean/Monoidal.lean` for review by Neil Ghani

July 2026

Abstract

This document presents every definition and theorem *statement* from the Lean 4 module `Containers.Monoidal` in three parallel forms: the original Lean, a readable Haskell rendering, and precise mathematical English. Proofs are omitted — they are machine-checked by Lean. The purpose is to let a human verifier confirm that the formalisation captures the intended mathematics.

The file proves that $(\mathbf{Cont}, \triangleleft, I)$ is a monoidal category, by supplying the bifunctoriality of $\triangleleft$ on morphisms, the naturality of the structural isomorphisms, the pentagon, and the triangle. All laws hold by `rfl` (definitional equality), since the position fibres line up definitionally. A minimal Mathlib-free `MonoidalCategory` typeclass is also defined, and the instance `ContMonoidal` assembles the whole.

Contents

1 Overview **2**

2 Bifunctoriality of $\triangleleft$ on morphisms **2**
- 2.1 Definition: `Container.seq₂` . . . 2
- 2.2 Definition: `Container.whiskerLeft` . . . 2
- 2.3 Definition: `Container.whiskerRight` . . . 3

3 Bifunctor laws for $\triangleleft$ **3**
- 3.1 Theorem: `Container.seq₂_id` . . . 3
- 3.2 Theorem: `Container.seq₂_comp` . . . 3

4 Naturality of the structural isomorphisms **4**
- 4.1 Theorem: `Container.associator_naturality` . . . 4
- 4.2 Theorem: `Container.leftUnitor_naturality` . . . 4
- 4.3 Theorem: `Container.rightUnitor_naturality` . . . 4

5 Coherence **5**
- 5.1 Theorem: `Container.pentagon` . . . 5
- 5.2 Theorem: `Container.triangle` . . . 5

6 The monoidal category typeclass **5**
- 6.1 Structure: `CatIso` . . . 5
- 6.2 Class: `MonoidalCategory` . . . 6
- 6.3 Definition: `ContainerIso.toCatIso` . . . 7
- 6.4 Instance: `ContMonoidal` . . . 7

1 Overview

Building on `Containers.Sequential`, which defines $\triangleleft$, the unit I, and the associator/unitors as isomorphisms, this file supplies the remaining ingredients to certify $(\mathbf{Cont}, \triangleleft, I)$ as a monoidal category.

Key insight: the associator's shape map is *currying* and its position map is Σ-reassociation. Both pentagon legs reduce to the same flattening of a fourfold composite, and every map is the identity on the underlying flat tuple.

2 Bifunctoriality of $\triangleleft$ on morphisms

2.1 Definition: `Container.seq₂`

Definition 2.1 (Bifunctorial action of $\triangleleft$)**.** Given morphisms $\mu : G \Rightarrow G'$ and $\nu : F \Rightarrow F'$, the **action of $\triangleleft$ on the pair** is the morphism $\mathtt{seq}_2\,\mu\,\nu : G \triangleleft F \Rightarrow G' \triangleleft F'$. On shapes, a shape (t, f) of $G \triangleleft F$ — an outer shape t and an inner-shape assignment $f : P_G(t) \to S_F$ — maps to $(\mu(t),\ \nu \circ f \circ \mu_t^\sharp)$, where $\mu_t^\sharp$ is the backward position map of μ at t. On positions: both backward position maps are applied componentwise.

Lean.

```
def Container.seq2 {G G' F F' : Container}
    (mu : ContainerMorphism G G') (nu : ContainerMorphism F F') :
    ContainerMorphism (G seq F) (G' seq F') where
  onShapes := fun tf ⇒
    ⟨mu.onShapes tf.1, fun q' ⇒ nu.onShapes (tf.2 (mu.onPos tf.1 q'))
      ⟩
  onPos := fun tf qr ⇒
    ⟨mu.onPos tf.1 qr.1, nu.onPos (tf.2 (mu.onPos tf.1 qr.1)) qr.2⟩
```

Haskell.

```
-- G seq F has Shape = (t, f) where t :: Shape G, f :: Pos G t →
   Shape F
-- seq2 mu nu : ContainerMorphism (G `seq` F) (G' `seq` F')
seq2 :: Morph g p g' p' → Morph f q f' q'
     → Morph (g `Seq` f) (p, q) (g' `Seq` f') (p', q')
seq2 mu nu = Morph
  { onShapes = \(t, f) → (onShapes mu t, \q' → onShapes nu (f (onPos
      mu t q')))
  , onPos    = \(t, f) (qr1, qr2) →
      (onPos mu t qr1, onPos nu (f (onPos mu t qr1)) qr2) }
```

Fidelity note. The Lean definition uses dependent types: `tf.2` is a function from $P_G(\mathtt{tf.1})$, whose type depends on the shape `tf.1`. The Haskell rendering non-dependently pairs the components.

2.2 Definition: `Container.whiskerLeft`

Definition 2.2 (Left whiskering)**. Left whiskering** $G \triangleleft \nu : G \triangleleft F \Rightarrow G \triangleleft F'$ acts by ν on the inner container, leaving the outer container G fixed:

$$G \triangleleft \nu \;=\; \mathtt{seq}_2\,(\mathrm{id}_G)\,\nu.$$

Lean.

```
def Container.whiskerLeft (G : Container) {F F' : Container}
    (nu : ContainerMorphism F F') : ContainerMorphism (G seq F) (G seq
        F') :=
  Container.seq2 (ContainerMorphism.id G) nu
```

Haskell.
```
whiskerLeft :: Proxy g → Morph f q f' q'
            → Morph (g `Seq` f) (p, q) (g `Seq` f') (p, q')
whiskerLeft _ nu = seq2 idMorph nu
```

2.3 Definition: `Container.whiskerRight`

Definition 2.3 (Right whiskering)**. Right whiskering** $\mu \triangleleft F : G \triangleleft F \Rightarrow G' \triangleleft F$ acts by μ on the outer container, leaving the inner container F fixed:

$$\mu \triangleleft F \;=\; \mathsf{seq}_2\, \mu\, (\mathrm{id}_F).$$

Lean.
```
def Container.whiskerRight {G G' : Container} (mu : ContainerMorphism
   G G')
    (F : Container) : ContainerMorphism (G seq F) (G' seq F) :=
  Container.seq2 mu (ContainerMorphism.id F)
```

Haskell.
```
whiskerRight :: Morph g p g' p' → Proxy f
             → Morph (g `Seq` f) (p, q) (g' `Seq` f) (p', q)
whiskerRight mu _ = seq2 mu idMorph
```

3 Bifunctor laws for $\triangleleft$

3.1 Theorem: $\mathtt{Container.seq_2_id}$

Theorem 3.1 (Bifunctoriality: identity)**.** seq_2 preserves identities:

$$\mathsf{seq}_2\, (\mathrm{id}_G)\, (\mathrm{id}_F) \;=\; \mathrm{id}_{G \triangleleft F}.$$

Lean.
```
theorem Container.seq2_id (G F : Container) :
    Container.seq2 (ContainerMorphism.id G) (ContainerMorphism.id F)
      = ContainerMorphism.id (G seq F) := rfl
```

Haskell.
```
-- seq2 idMorph idMorph = idMorph
```

3.2 Theorem: $\mathtt{Container.seq_2_comp}$

Theorem 3.2 (Bifunctoriality: interchange)**.** seq_2 preserves composition (interchange / bifunctoriality):

$$\mathsf{seq}_2\, (\mu \cdot \mu')\, (\nu \cdot \nu') \;=\; \mathsf{seq}_2\, \mu\, \nu \;\cdot\; \mathsf{seq}_2\, \mu'\, \nu'.$$

Lean.
```
theorem Container.seq2_comp {G G' G'' F F' F'' : Container}
    (mu : ContainerMorphism G G') (mu' : ContainerMorphism G' G'')
    (nu : ContainerMorphism F F') (nu' : ContainerMorphism F' F'') :
    Container.seq2 (mu.comp mu') (nu.comp nu')
      = (Container.seq2 mu nu).comp (Container.seq2 mu' nu') := rfl
```

Haskell.
```
-- seq2 (mu `compM` mu') (nu `compM` nu')
--   = seq2 mu nu `compM` seq2 mu' nu'
```

4 Naturality of the structural isomorphisms

4.1 Theorem: `Container.associator_naturality`

Theorem 4.1 (Naturality of the associator)**.** The associator α is natural in all three arguments: for all morphisms $f_i : X_i \Rightarrow Y_i$,

$$(\mathsf{seq}_2\,(\mathsf{seq}_2\,f_1\,f_2)\,f_3) \cdot \alpha_{Y_1,Y_2,Y_3} \;=\; \alpha_{X_1,X_2,X_3} \cdot \mathsf{seq}_2\,f_1\,(\mathsf{seq}_2\,f_2\,f_3).$$

Lean.

```
theorem Container.associator_naturality {X1 Y1 X2 Y2 X3 Y3 : Container
    }
    (f1 : ContainerMorphism X1 Y1) (f2 : ContainerMorphism X2 Y2)
    (f3 : ContainerMorphism X3 Y3) :
    (Container.seq2 (Container.seq2 f1 f2) f3).comp
        (Container.associator Y1 Y2 Y3).hom
      = (Container.associator X1 X2 X3).hom.comp
          (Container.seq2 f1 (Container.seq2 f2 f3)) :=
  rfl
```

Haskell.

```
-- (seq2 (seq2 f1 f2) f3) `compM` assocHom y1 y2 y3
--   = assocHom x1 x2 x3 `compM` seq2 f1 (seq2 f2 f3)
```

4.2 Theorem: `Container.leftUnitor_naturality`

Theorem 4.2 (Naturality of the left unitor)**.** The left unitor λ is natural: for $f : X \Rightarrow Y$,

$$(I \triangleleft f) \cdot \lambda_Y \;=\; \lambda_X \cdot f.$$

Lean.

```
theorem Container.leftUnitor_naturality {X Y : Container}
    (f : ContainerMorphism X Y) :
    (Container.whiskerLeft Container.I f).comp (Container.leftUnitor Y
       ).hom
      = (Container.leftUnitor X).hom.comp f := rfl
```

Haskell.

```
-- whiskerLeft I f `compM` leftUnitorHom y
--   = leftUnitorHom x `compM` f
```

4.3 Theorem: `Container.rightUnitor_naturality`

Theorem 4.3 (Naturality of the right unitor)**.** The right unitor ρ is natural: for $f : X \Rightarrow Y$,

$$(f \triangleleft I) \cdot \rho_Y \;=\; \rho_X \cdot f.$$

Lean.

```
theorem Container.rightUnitor_naturality {X Y : Container}
    (f : ContainerMorphism X Y) :
    (Container.whiskerRight f Container.I).comp (Container.rightUnitor
        Y).hom
      = (Container.rightUnitor X).hom.comp f := rfl
```

Haskell.

```
-- whiskerRight f I `compM` rightUnitorHom y
--   = rightUnitorHom x `compM` f
```

5 Coherence

5.1 Theorem: `Container.pentagon`

Theorem 5.1 (Pentagon identity)**.** The two ways of re-associating $((C_1 \triangleleft C_2) \triangleleft C_3) \triangleleft C_4$ to $C_1 \triangleleft (C_2 \triangleleft (C_3 \triangleleft C_4))$ coincide:

$$(\alpha_{C_1,C_2,C_3} \triangleleft C_4) \cdot \alpha_{C_1,C_2 \triangleleft C_3,C_4} \cdot (C_1 \triangleleft \alpha_{C_2,C_3,C_4}) \;=\; \alpha_{C_1 \triangleleft C_2,C_3,C_4} \cdot \alpha_{C_1,C_2,C_3 \triangleleft C_4}.$$

Lean.

```
theorem Container.pentagon (C1 C2 C3 C4 : Container) :
    ((Container.whiskerRight (Container.associator C1 C2 C3).hom C4).
       comp
        (Container.associator C1 (C2 seq C3) C4).hom).comp
        (Container.whiskerLeft C1 (Container.associator C2 C3 C4).hom)
      = (Container.associator (C1 seq C2) C3 C4).hom.comp
        (Container.associator C1 C2 (C3 seq C4)).hom := rfl
```

Haskell.

```
-- (whiskerRight assoc123 C4 `compM` assoc1_23_4 `compM`
--   whiskerLeft C1 assoc234)
--   = assoc12_3_4 `compM` assoc1_2_34
```

5.2 Theorem: `Container.triangle`

Theorem 5.2 (Triangle identity)**.** The associator through the unit composed with the inner left unitor equals the outer right unitor:

$$\alpha_{C_1,I,C_2} \cdot (C_1 \triangleleft \lambda_{C_2}) \;=\; \rho_{C_1} \triangleleft C_2,$$

as morphisms $(C_1 \triangleleft I) \triangleleft C_2 \Rightarrow C_1 \triangleleft C_2$.

Lean.

```
theorem Container.triangle (C1 C2 : Container) :
    (Container.associator C1 Container.I C2).hom.comp
        (Container.whiskerLeft C1 (Container.leftUnitor C2).hom)
      = Container.whiskerRight (Container.rightUnitor C1).hom C2 :=
         rfl
```

Haskell.

```
-- assocHom c1 I c2 `compM` whiskerLeft c1 (leftUnitorHom c2)
--   = whiskerRight (rightUnitorHom c1) c2
```

6 The monoidal category typeclass

6.1 Structure: `CatIso`

Definition 6.1 (Isomorphism in a category)**.** A **categorical isomorphism** between objects X and Y in a category $\mathcal{C}$ consists of a forward morphism `hom` and a backward morphism `inv` satisfying both round-trip conditions: $\texttt{hom} \cdot \texttt{inv} = \mathrm{id}_X$ and $\texttt{inv} \cdot \texttt{hom} = \mathrm{id}_Y$.

Lean.

```
structure CatIso {Obj : Type u} [Category Obj] (X Y : Obj) where
  hom : Category.Hom X Y
  inv : Category.Hom Y X
  hom_inv_id : Category.comp hom inv = Category.id X
  inv_hom_id : Category.comp inv hom = Category.id Y
```

Haskell.

```haskell
data CatIso cat x y = CatIso
  { catHom    :: cat x y
  , catInv    :: cat y x
  , homInvId  :: catHom `comp` catInv == idC x  -- propositional
  , invHomId  :: catInv `comp` catHom == idC y
  }
```

6.2 Class: `MonoidalCategory`

Definition 6.2 (Monoidal category (Mathlib-free))**.** A **monoidal category** extends `Category` with:

- a tensor on objects $\otimes : \mathrm{Obj} \to \mathrm{Obj} \to \mathrm{Obj}$,
- a bifunctorial tensor on morphisms,
- a unit object $\mathbf{1}$,
- associator and unitor isomorphisms,
- naturality of the associator and unitors, and
- the pentagon and triangle coherence laws.

Lean.

```
class MonoidalCategory (Obj : Type u) extends Category.{u, v} Obj
    where
  tensorObj     : Obj → Obj → Obj
  tensorHom     : {X1 Y1 X2 Y2 : Obj} →
    Hom X1 Y1 → Hom X2 Y2 → Hom (tensorObj X1 X2) (tensorObj Y1 Y2)
  tensorHom_id : ∀ (X Y : Obj),
    tensorHom (id X) (id Y) = id (tensorObj X Y)
  tensorHom_comp : ∀ {X1 Y1 Z1 X2 Y2 Z2 : Obj}
    (f1 : Hom X1 Y1) (g1 : Hom Y1 Z1) (f2 : Hom X2 Y2) (g2 : Hom Y2 Z2
       ),
    tensorHom (comp f1 g1) (comp f2 g2)
      = comp (tensorHom f1 f2) (tensorHom g1 g2)
  tensorUnit        : Obj
  associator        : (X Y Z : Obj) →
    CatIso (tensorObj (tensorObj X Y) Z) (tensorObj X (tensorObj Y Z))
  leftUnitor        : (X : Obj) → CatIso (tensorObj tensorUnit X) X
  rightUnitor       : (X : Obj) → CatIso (tensorObj X tensorUnit) X
  associator_naturality : ∀ {X1 Y1 X2 Y2 X3 Y3 : Obj}
    (f1 : Hom X1 Y1) (f2 : Hom X2 Y2) (f3 : Hom X3 Y3),
    comp (tensorHom (tensorHom f1 f2) f3) (associator Y1 Y2 Y3).hom
      = comp (associator X1 X2 X3).hom (tensorHom f1 (tensorHom f2 f3)
         )
  leftUnitor_naturality : ∀ {X Y : Obj} (f : Hom X Y),
    comp (tensorHom (id tensorUnit) f) (leftUnitor Y).hom
      = comp (leftUnitor X).hom f
  rightUnitor_naturality : ∀ {X Y : Obj} (f : Hom X Y),
    comp (tensorHom f (id tensorUnit)) (rightUnitor Y).hom
      = comp (rightUnitor X).hom f
  pentagon : ∀ (W X Y Z : Obj),
    comp (comp (tensorHom (associator W X Y).hom (id Z))
               (associator W (tensorObj X Y) Z).hom)
         (tensorHom (id W) (associator X Y Z).hom)
      = comp (associator (tensorObj W X) Y Z).hom
             (associator W X (tensorObj Y Z)).hom
```

```
  triangle : ∀ (X Y : Obj),
    comp (associator X tensorUnit Y).hom
         (tensorHom (id X) (leftUnitor Y).hom)
      = tensorHom (rightUnitor X).hom (id Y)
```

Haskell.

```haskell
class Category cat ⇒ MonoidalCategory cat where
  tensorObj  :: obj → obj → obj
  tensorHom  :: cat x1 y1 → cat x2 y2
             → cat (tensorObj x1 x2) (tensorObj y1 y2)
  tensorUnit :: obj
  assoc      :: CatIso cat (tensorObj (tensorObj x y) z)
                           (tensorObj x (tensorObj y z))
  leftUnit   :: CatIso cat (tensorObj tensorUnit x) x
  rightUnit  :: CatIso cat (tensorObj x tensorUnit) x
  -- plus naturality, pentagon, triangle laws as proof obligations
```

6.3 Definition: `ContainerIso.toCatIso`

Definition 6.3 (Reinterpretation of container isomorphisms)**.** A `ContainerIso` (from `Cont.lean`) can be viewed as a `CatIso` for the category `Container`: the fields coincide definitionally.

Lean.

```
def ContainerIs ∘ toCatIso {C D : Container} (i : ContainerIso C D) :
    CatIso C D :=
  ⟨i.hom, i.inv, i.hom_inv, i.inv_hom⟩
```

Haskell.

```haskell
toCatIso :: ContainerIso c d → CatIso ContainerMorph c d
toCatIso i = CatIso (isoHom i) (isoInv i) (homInv i) (invHom i)
```

6.4 Instance: `ContMonoidal`

Definition 6.4 ($(\mathbf{Cont}, \triangleleft, I)$ is monoidal)**.** The sequential operator $\triangleleft$ with morphism action $\mathtt{seq}_2$, unit container I, and structural isomorphisms from `Sequential.lean` constitute a monoidal category. Every field — the two functor laws, three naturality squares, pentagon, triangle — is discharged by `rfl`.

Lean.

```
instance ContMonoidal : MonoidalCategory Container where
  tensorObj          := Container.seq
  tensorHom          := Container.seq2
  tensorHom_id       := Container.seq2_id
  tensorHom_comp     := fun f1 g1 f2 g2 ⇒ Container.seq2_comp f1 g1 f2
      g2
  tensorUnit         := Container.I
  associator         := fun X Y Z ⇒ (Container.associator X Y Z).
     toCatIso
  leftUnitor         := fun X ⇒ (Container.leftUnitor X).toCatIso
  rightUnitor        := fun X ⇒ (Container.rightUnitor X).toCatIso
  associator_naturality :=
    fun f1 f2 f3 ⇒ Container.associator_naturality f1 f2 f3
  leftUnitor_naturality  := fun f ⇒ Container.leftUnitor_naturality f
  rightUnitor_naturality := fun f ⇒ Container.rightUnitor_naturality
     f
  pentagon           := Container.pentagon
  triangle           := Container.triangle
```

Haskell.

```haskell
-- The sequential operator `seq` (written `triangleleft`) gives Cont
-- the structure of a monoidal category, with:
--   tensorObj = seq
--   tensorHom = seq2
--   unit      = I
-- All laws hold definitionally.
instance MonoidalCategory ContainerMorph where ...
```

Fidelity note. In Lean, only one `MonoidalCategory Container` *instance* can be active at a time; this is the canonical one. The other three monoidal structures on `Container` (coproduct $+$, product $\times$, Dirichlet $\otimes$) are given as named `def`s in `FourMonoidal.lean` / `Dirichlet.lean`, to avoid instance clashes.